App Dev Project Report

1. Student Details

Name: Bendalam Venkata Shivani
Roll Number: 22f3001283
Email: 22f3001283@ds.study.iitm.ac.in

About Me: I am a final year student at Gayatri Vidya Parishad and about to complete my Diploma's in this term from IITM Online BS Degree Program. I am an ML and Full-Stack Developer enthusiast. I consider analytical and logical thinking as my strengths.

---

2. Project Details

Project Title: Trekking Management Application

Problem Statement:

It's a role-based trekking management platform where admins create and control treks and assign them to staff, staff manage their assigned treks, and users can browse, book, and track their treks.

Approach:

There are three types of users with different functionalities in this project. Instead of separate tables for each, I used a single User model with a role field. For login, I used JWT tokens, and every backend route checks the role before granting access. On the Vue side, the router also checks the user's role and redirects them to their own dashboard. I started with the database models, then implemented authentication and roles, followed by treks, bookings and booking history, and added Celery jobs and Redis caching at the end.

---

3. AI/LLM Declaration

I used **Claude (Anthropic)** in a limited, assistive role during development.
It helped with migrating frontend styling from custom CSS to Bootstrap utility classes, and resolving form validation edge cases in Vue watchers. It also helped set up Chart.js for statistics dashboards, implement Flask API endpoints that I had already defined and structured, and debug a Celery Beat time zone scheduling issue (IST/UTC handling).
The extent of AI/LLM usage is around 20–22%, limited to frontend styling, specific implementation help, and debugging assistance on issues I had already identified.

All database design, application architecture, debugging, and final implementation logic were done manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask (Flask-RESTful, JWT-Extended, CORS)	Core backend web framework and REST API
SQLAlchemy + SQLite	Database and ORM for storing application data
Werkzeug Security	Password hashing for user accounts
Flask-Mail	Sending email notifications
Flask-Caching + Redis + Celery	Caching, background jobs, and scheduled tasks
Vue 3 (+ Router, Axios)	Frontend framework, routing, and API calls
Bootstrap 5	Frontend styling and responsive design
Chart.js	Visualizing statistics and analytics trends
CSV (Python module)	Exporting booking/participant data

---

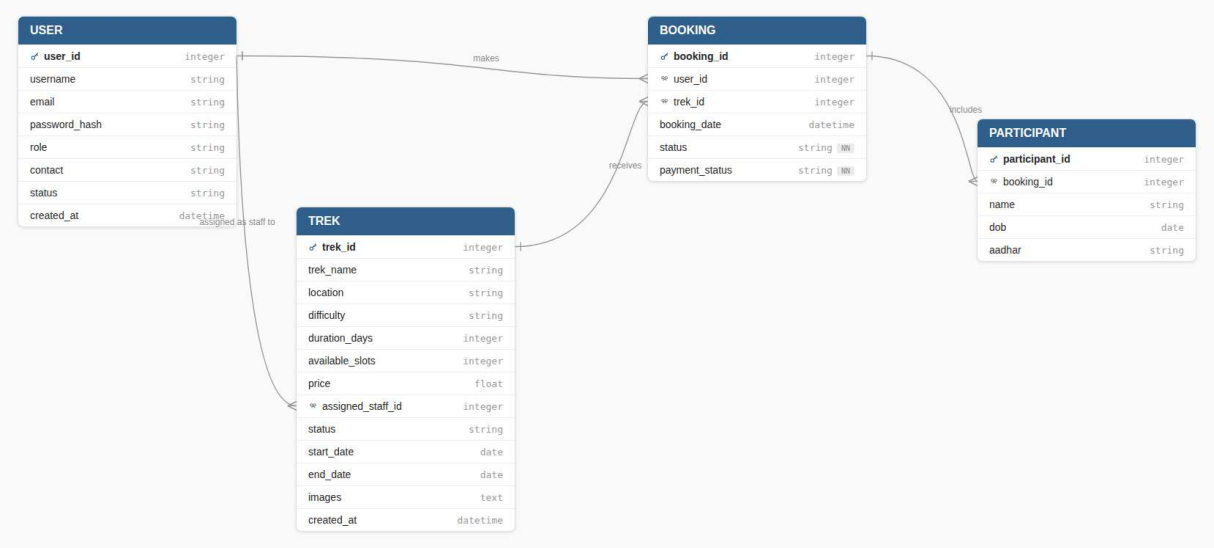
5. Database Schema / ER Diagram

Tables and Relationships

1. **User** — stores user profile and role details (user_id, username, email, password_hash, role, contact, status, created_at)
2. **Trek** — stores trek details created by admin (trek_id, trek_name, location, difficulty, duration_days, available_slots, price, assigned_staff_id, status, start_date, end_date, images, created_at)
3. **Booking** — records a user's trek booking (booking_id, user_id, trek_id, booking_date, status, payment_status)
4. **Participant** — stores participant details for a booking (participant_id, booking_id, name, dob, aadhar)

Relationships:

- One-to-Many → User → Booking (a user can make many bookings)
- One-to-Many → User → Trek (a staff user can be assigned to many treks)
- One-to-Many → Trek → Booking (a trek can have many bookings)
- One-to-Many → Booking → Participant (a booking can include many participants)



---

6. API Resource Endpoints

Endpoint	Method	Description
/login	POST	Authenticate user and return JWT token
/signup	POST	Register a new user
/public/stats	GET	Public trekking stats for homepage (no auth)
/treks	GET	List all treks
/treks	POST	Create a new trek (admin)
/treks/<trek_id>	GET	Fetch a single trek
/treks/<trek_id>	PUT	Update trek details/status (admin)
/treks/<trek_id>	DELETE	Delete a trek (admin)
/bookings	POST	Create a booking with participants (user)
/bookings	GET	List bookings (role-based scope)

Endpoint	Method	Description
/bookings/<booking_id>	GET	Fetch a single booking
/bookings/<booking_id>	DELETE	Cancel a booking
/staff/treks	GET	List treks assigned to logged-in staff
/treks/<trek_id>/staff	PUT	Staff updates slots/status/completion
/staff/profile	GET	Fetch staff's own profile
/staff/profile	PUT	Update staff's own profile
/user/bookings	GET	User's latest booking per trek
/user/treks	GET	List Open/Approved treks
/user/profile	GET	Fetch user's own profile
/user/profile	PUT	Update user's own profile
/users/pending	GET	List blacklisted users (admin)
/users/<user_id>	PUT	Update user status (admin)
/users	GET	List all users (admin, staff)
/staff	POST	Create a new staff account (admin)
/admin/stats	GET	Admin KPIs and analytics charts
/export/booking-history	POST	Trigger CSV export of booking history
/export/status/<task_id>	GET	Poll export task status
/export/download/<filename>	GET	Download completed export file
/export/trek-participants/<trek_id>	POST	Trigger CSV export of trek participants

---

7. Architecture and Features (optional)

Architecture Overview:

- **backend/app.py –** main Flask application entry point, containing all API resources/routes
- **backend/models.py –** database models and relationships using SQLAlchemy
- **backend/main.py –** application runner/startup script
- **backend/celerybeat-schedule –** Celery Beat schedule file for periodic background jobs (reminders, monthly reports)
- **frontend/src/views/ –** role-based page components, organized into Admin/, Staff/, and User/ subfolders, plus shared pages like Home.vue, LoginView.vue, SignupView.vue
- **frontend/src/components/ –** shared/reusable UI components (Booking.vue, Trek.vue, BookingHistory.vue, role-specific navbars)
- **frontend/src/router/index.js**– Vue Router configuration with role-based route guards

Implemented Features (Core Requirements):

- User self-registration and login; Admin/Staff login only (no self-registration)
- Role-based access control using JWT, enforced on both backend routes and frontend route guards
- Admin: create/update/remove treks, add and manage staff, assign staff to treks, view/search users-staff-treks, blacklist/deactivate accounts
- Staff: view assigned treks, update slots/status, manage participant list, mark treks completed
- User: search/filter treks by difficulty/location/duration, book treks, view booking status and history, edit profile
- Booking safeguards: prevent overbooking beyond available slots, prevent duplicate bookings, allow booking only when trek status is Open

Additional Features:

- Celery background jobs: daily trek reminder emails, monthly HTML activity report emailed to Admin, async CSV export of booking/participant history
- Redis caching with expiry on frequently accessed endpoints (trek listings, stats, user bookings)
- Chart.js analytics dashboard (booking trends, popular treks, monthly participation)
- Public landing page with read-only trekking statistics (pre-login)
- Responsive UI built with Bootstrap

---

8. Video Presentation

Drive Link:
https://drive.google.com/file/d/1IBTX-eTR7SaCKqsF6yk83BRaozjbiuSO/view?usp=sharing